

SKILL WEEK – 5

Session - 9

TASK 1: To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation



```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_PATH 4096

int main() {
    char current_dir[MAX_PATH];
    char previous_dir[MAX_PATH];
    char input[MAX_PATH];

    printf("Session 8 - Directory Navigation\n");
    printf("Enter directory path to change:\n");
    printf("myshell> ");

    if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
        perror("getcwd failed");
        return 1;
    }

    strcpy(previous_dir, current_dir);

    if (fgets(input, sizeof(input), stdin) == NULL) {
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strlen(input) == 0) {
        printf("Empty path.\n");
        return 0;
    }

    if (strcmp(input, "..") == 0) {
        if (chdir("..") != 0) {
            perror("Directory change failed");
            return 1;
        }
    }
    else if (strcmp(input, "previous") == 0) {
        if (chdir(previous_dir) != 0) {
            perror("Directory change failed");
            return 1;
        }
    }
    else {
        if (chdir(input) != 0) {
            perror("Directory change failed");
            printf("Invalid path: %s\n", input);
            return 1;
        }
    }
}
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1

strcpy(previous_dir, current_dir);

if (fgets(input, sizeof(input), stdin) == NULL) {
    return 1;
}

input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0) {
    printf("Empty path.\n");
    return 0;
}

if (strcmp(input, "..") == 0) {
    if (chdir("..") != 0) {
        perror("Directory change failed");
        return 1;
    }
}
else if (strcmp(input, "previous") == 0) {
    if (chdir(previous_dir) != 0) {
        perror("Directory change failed");
        return 1;
    }
}
else {
    if (chdir(input) != 0) {
        perror("Directory change failed");
        printf("Invalid path: %s\n", input);
        return 1;
    }
}

if (getcwd(current_dir, sizeof(current_dir)) == NULL) {
    perror("getcwd failed");
    return 1;
}

printf("\nPrevious directory:\n");
printf("%s\n", previous_dir);

printf("\nCurrent directory:\n");
printf("%s\n", current_dir);

printf("\nNavigation validation:\n");
printf("Directory changed successfully.\n");
printf("Working directory updated.\n");

return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:~$ nano session9_task1.c
keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc session9_task1.c -o session9_task1
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./session9_task1
Session 8 - Directory Navigation
Enter directory path to change:
myshell> /tmp

Previous directory:
/home/keerthi_priya

Current directory:
/tmp

Navigation validation:
Directory changed successfully.
Working directory updated.
keerthi_priya@LAPTOP-OSGVCE7A:~$ |
```

TASK 2: TO Identify Built-ins, Create Dispatch Table, Execute In- Process Commands, Handle Invalid Commands, Maintain State, Test Dispatch Logic

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 100

void cmd_help() {
    printf("Dispatching built-in: help\n\n");
    printf("Available built-in commands:\n");
    printf("  help  - Display available commands\n");
    printf("  pwd   - Display current directory\n");
    printf("  cd    - Change directory\n");
    printf("  exit  - Exit the shell\n");
}

void cmd_pwd() {
    char cwd[MAX_INPUT];

    printf("Dispatching built-in: pwd\n");

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("getcwd failed");
}

void cmd_cd(char *path) {
    printf("Dispatching built-in: cd\n");

    if (path == NULL) {
        printf("cd: missing argument\n");
        return;
    }

    if (chdir(path) != 0)
        perror("cd failed");
}

void cmd_exit() {
    printf("Dispatching built-in: exit\n");
}
```

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1

void cmd_exit() {
    printf("Dispatching built-in: exit\n");
    printf("Exiting shell...\n");
    exit(0);
}

int main() {
    char input[MAX_INPUT];
    char *command;
    char *argument;

    printf("Session 9 - Built-in Command Dispatcher\n");
    printf("Type 'help' to see available commands.\n\n");

    while (1) {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        command = strtok(input, " ");
        argument = strtok(NULL, " ");

        if (command == NULL)
            continue;

        if (strcmp(command, "help") == 0) {
            cmd_help();
        }
        else if (strcmp(command, "pwd") == 0) {
            cmd_pwd();
        }
        else if (strcmp(command, "cd") == 0) {
            cmd_cd(argument);
        }
        else if (strcmp(command, "exit") == 0) {
            cmd_exit();
        }
    }
}
```

```
keerthi_priya@LAPTOP-OSGV x + v
GNU nano 8.7.1

char *argument;

printf("Session 9 - Built-in Command Dispatcher\n");
printf("Type 'help' to see available commands.\n\n");

while (1) {
    printf("myshell> ");
    fflush(stdout);

    if (fgets(input, sizeof(input), stdin) == NULL)
        break;

    input[strcspn(input, "\n")] = '\0';

    command = strtok(input, " ");
    argument = strtok(NULL, " ");

    if (command == NULL)
        continue;

    if (strcmp(command, "help") == 0) {
        cmd_help();
    }
    else if (strcmp(command, "pwd") == 0) {
        cmd_pwd();
    }
    else if (strcmp(command, "cd") == 0) {
        cmd_cd(argument);
    }
    else if (strcmp(command, "exit") == 0) {
        cmd_exit();
    }
    else {
        printf("Invalid command: %s\n", command);
        printf("Type 'help' to see available built-ins.\n");
    }
}

return 0;
}
```

```

keerthi_priya@LAPTOP-OSGVCE7A:~$ nano session9_task2.c

keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc session9_task2.c -o session9_task2
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./session9_task2
Session 9 - Built-in Command Dispatcher
Type 'help' to see available commands.

myshell> help
Dispatching built-in: help

Available built-in commands:
  help - Display available commands
  pwd  - Display current directory
  cd   - Change directory
  exit - Exit the shell
myshell> pwd
Dispatching built-in: pwd
/home/keerthi_priya
myshell> cd /tmp
Dispatching built-in: cd
myshell> pwd
Dispatching built-in: pwd
/tmp
myshell> hello
Invalid command: hello
Type 'help' to see available built-ins.
myshell> exit
Dispatching built-in: exit
Exiting shell...
keerthi_priya@LAPTOP-OSGVCE7A:~$ |

```

Session – 10

TASK 1: To Retrieve Current Directory, Display path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior

```

GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_PATH 4096
#define MAX_INPUT 100

int main()
{
    char current_dir[MAX_PATH];
    char input[MAX_INPUT];

    printf("Session 9 - Current Directory and Exit\n");

    /* Retrieve current directory */
    if (getcwd(current_dir, sizeof(current_dir)) == NULL)
    {
        perror("getcwd failed");
        return 1;
    }

    printf("Current Directory: %s\n", current_dir);

    printf("\nEnter a command:\n");
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        printf("Input error.\n");
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    /* Process exit request */
    if (strcmp(input, "exit") == 0)
    {
        printf("\nExit request received.\n");

        /* Save state */
        printf("Saving current directory: %s\n", current_dir);

        /* Cleanup resources */
        printf("Cleaning up resources...\n");
    }
}

```

```
keerthi_priya@LAPTOP-OSGV X + v
GNU nano 8.7.1
{
    perror("getcwd failed");
    return 1;
}

printf("Current Directory: %s\n", current_dir);

printf("\nEnter a command:\n");
printf("myshell> ");

if (fgets(input, sizeof(input), stdin) == NULL)
{
    printf("Input error.\n");
    return 1;
}

input[strcspn(input, "\n")] = '\0';

/* Process exit request */
if (strcmp(input, "exit") == 0)
{
    printf("\nExit request received.\n");

    /* Save state */
    printf("Saving current directory: %s\n", current_dir);

    /* Cleanup resources */
    printf("Cleaning up resources...\n");

    printf("Shell exited successfully.\n");
    return 0;
}

printf("\nCommand entered: %s\n", input);

/* Verify current directory */
if (getcwd(current_dir, sizeof(current_dir)) != NULL)
{
    printf("Verified Current Directory: %s\n", current_dir);
}

printf("Program completed successfully.\n");

return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:~$ nano session10_task1.c
keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc session10_task1.c -o session10_task1
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./session10_task1
Session 9 - Current Directory and Exit
Current Directory: /home/keerthi_priya

Enter a command:
myshell> exit

Exit request received.
Saving current directory: /home/keerthi_priya
Cleaning up resources...
Shell exited successfully.
keerthi_priya@LAPTOP-OSGVCE7A:~$ |
```

TASK 2: To Parse Export Syntax, Update Environment Variables, Validate Names, Handle Existing Variables, Export to Child Processes, Test Functionality

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 200
#define MAX_NAME 100
#define MAX_VALUE 100

int valid_name(const char *name)
{
    if (name[0] == '\0')
        return 0;

    if (!isalpha((unsigned char)name[0]) && name[0] != '_')
        return 0;

    for (int i = 1; name[i] != '\0'; i++)
    {
        if (!isalnum((unsigned char)name[i]) && name[i] != '_')
            return 0;
    }

    return 1;
}

int main()
{
    char input[MAX_INPUT];
    char name[MAX_NAME];
    char value[MAX_VALUE];

    printf("Session 9 - Export Environment Variables\n");

    printf("Enter export command:\n");
    printf("myshell> ");

    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        printf("Input error.\n");
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strncmp(input, "export ", 7) != 0)
    {
        printf("Invalid syntax. Use: export NAME=value\n");
        return 1;
    }

    char *assignment = input + 7;
```

```
char *assignment = input + 7;
char *equals = strchr(assignment, '=');

if (equals == NULL)
{
    printf("Invalid export syntax.\n");
    printf("Use: export NAME=value\n");
    return 1;
}

int name_length = equals - assignment;

if (name_length >= MAX_NAME)
{
    printf("Variable name too long.\n");
    return 1;
}

strncpy(name, assignment, name_length);
name[name_length] = '\0';

strcpy(value, equals + 1);

if (!valid_name(name))
{
    printf("Invalid variable name: %s\n", name);
    return 1;
}

char *old_value = getenv(name);

if (old_value != NULL)
{
    printf("Existing variable found.\n");
    printf("Old value: %s\n", old_value);
}
else
{
    printf("New environment variable.\n");
}

if (setenv(name, value, 1) != 0)
{
    perror("setenv failed");
    return 1;
}

printf("\nEnvironment variable updated successfully.\n");
printf("Variable name : %s\n", name);
printf("Variable value: %s\n", getenv(name));
```

```
keerthi_priya@LAPTOP-OSGV × + v
GNU nano 8.7.1
    }
    return 1;
}

int name_length = equals - assignment;

if (name_length >= MAX_NAME)
{
    printf("Variable name too long.\n");
    return 1;
}

strncpy(name, assignment, name_length);
name[name_length] = '\0';

strcpy(value, equals + 1);

if (!valid_name(name))
{
    printf("Invalid variable name: %s\n", name);
    return 1;
}

char *old_value = getenv(name);

if (old_value != NULL)
{
    printf("Existing variable found.\n");
    printf("Old value: %s\n", old_value);
}
else
{
    printf("New environment variable.\n");
}

if (setenv(name, value, 1) != 0)
{
    perror("setenv failed");
    return 1;
}

printf("\nEnvironment variable updated successfully.\n");
printf("Variable name : %s\n", name);
printf("Variable value: %s\n", getenv(name));

printf("\nExport validation:\n");
printf("Variable name is valid.\n");
printf("Variable is available to child processes.\n");
printf("Export functionality completed successfully.\n");

return 0;
}
```

```
keerthi_priya@LAPTOP-OSGVCE7A:~$ nano session10_task2.c
keerthi_priya@LAPTOP-OSGVCE7A:~$ gcc session10_task2.c -o session10_task2
keerthi_priya@LAPTOP-OSGVCE7A:~$ ./session10_task2
Session 9 - Export Environment Variables
Enter export command:
myshell> export USERNAME=Keerthi
New environment variable.

Environment variable updated successfully.
Variable name : USERNAME
Variable value: Keerthi

Export validation:
Variable name is valid.
Variable is available to child processes.
Export functionality completed successfully.
keerthi_priya@LAPTOP-OSGVCE7A:~$ |
```

